W1 INTRODUCTION

DBMS role: controlled shared data, logical/physical independence, concurrency, recovery, security, integrity; schema=intension, instance=extension.
Models: conceptual ER/EER → logical relational → physical files/indexes. OLTP=short updates, high concurrency; OLAP=large scans/aggregates, star schemas.
Exam reflex: identify data, constraints, operations, anomalies, then choose ER/SQL/RA/FD/tx/index tool.
Design pipeline: UoD text → entities/attrs/relationships/cardinality/participation → relations+PK/FK → constraints+normalisation → workload/index/query plan.
Data independence: logical shields apps from schema changes via views; physical shields SQL from file/index changes.
Constraints: domain, key, entity integrity (PK non-null), referential integrity (FK references candidate key or nullable), semantic assertion/trigger.
Common traps: database is not just files; schema ≠ data; DBMS does not "normalise automatically"; a table can have many candidate keys but one chosen PK.

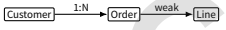
Term	Means	Exam cue
tuple	row	instance
relation	table	set-ish, no duplicate rows
attribute	column	domain typed
superkey	unique attrs	maybe non-minimal
candidate key	minimal superkey	choose PK

Mini ex: if Employee(empId,email,phone) has empId unique and email unique, both are candidate keys; PK is one chosen candidate key.
DBMS layers: query processor (parse/optimise/execute), storage manager (files/buffers/indexes), tx manager (locks/log/recovery), catalog (schemas/stats).
Catalog stats: table pages, tuples, distinct values, index height; optimiser estimates selectivity from these, so stale stats can cause poor plans.
Exam wording: "metadata" points to schema/catalog; "current contents" points to instance; "controlled redundancy" points to design/normalisation, not zero redundancy.
Mini ex: a view showing only Student(sid,name) gives security+logical independence; base table may also store phone/address.
Data dictionary: stores schemas, constraints, users, indexes, stats; SQL DDL updates catalog, DML updates data.
Workload thinking: read-heavy analytical workload favours indexes/materialised summaries; write-heavy OLTP penalises too many indexes.

W2 CONCEPTUAL DESIGN

ER basics: entity set, weak entity, relationship, attribute; key attr identifies entity; relationship attrs belong to association, not endpoint.
Cardinality: 1:1, 1:N, M:N; participation total/partial; min–max notation catches "must" vs "may".
Mapping: strong entity → relation; weak entity → relation incl owner key; 1:N put FK on N side; M:N create bridge; multi-valued attr create separate relation; composite attr flatten.
Weak entity key = owner PK + partial key. Deleting owner normally cascades or forbids weak rows.
EER: ISA specialization disjoint/overlap, total/partial; aggregation treats a relationship as higher-level entity when another relationship attaches to it.
ER reading: "at least one" = total participation/min 1; "at most one" = max 1; "exactly one" = min 1 max 1.

Pattern	Relational mapping	Risk
1:N	FK in N	wrong side FK
M:N	bridge(PK pair)	losing rel attrs
weak	owner FK in PK	missing cascade
ISA	one table or subtype tables	nulls/constraints



Mini ex: Order(orderNo), Line(lineNo,qty) weak under Order ⇒ Line(orderNo,lineNo,qty), PK(orderNo,lineNo), FK(orderNo).
Subtype mapping: single table w/type attr; superclass+subtype tables; subclass-only tables when total+disjoint. Tradeoff: nulls vs joins vs constraints.
Disjoint/overlap: disjoint=no entity in two subtypes; overlap=may be in many. Total=every supertype entity in at least one subtype; partial=optional subtype.
ER validation: read every noun/verb, mark ownership, decide if attr is multivalued/composite/derived, check all "must"/"may" phrases against min-cardinality.
Mini ex: Employee has many emails ⇒ Email(empId,email), PK(empId,email); do not store email1,email2,... columns.
Relationship attrs: dateAssigned in WorksOn(Employee,Project) belongs to bridge, because it depends on the pair.
Derived attr: age from DOB, totalDays from date range; store only if performance requires and maintain consistency.

W3 RELATIONAL MODEL

Relation schema: $R(A_1, \ldots, A_n)$ with domains; relation instance is set of tuples; keys impose uniqueness; FK stores logical pointer by value, not memory address.
Integrity: domain values valid; PK unique+not null; FK may be null unless declared NOT NULL; FK need not be unique unless modeling 1:1.
ER→RM sanity: every relation has PK; every M:N bridge has both FKs; every subtype FK references supertype; derived attrs usually not stored.
SQL DDL: PRIMARY KEY(A,B) means no duplicate pair and A,B are NOT NULL; UNIQUE(A,B) permits DBMS-specific NULL behavior.
NULL: unknown/missing/inapplicable; comparisons with NULL give UNKNOWN; WHERE keeps TRUE only; aggregate COUNT(*) counts rows, COUNT(col) ignores NULL, SUM ignores NULL.
3VL: TRUE OR UNKNOWN=TRUE; FALSE OR UNKNOWN=UNKNOWN; TRUE AND UNKNOWN=UNKNOWN; NOT UNKNOWN=UNKNOWN.

Operation	NULL behavior	Trap
=NULL	UNKNOWN	use IS NULL
AVG(x)	ignores NULL	not SUM/COUNT(*)
FK NULL	usually allowed	unless NOT NULL
NOT IN	NULL poisons	prefer NOT EXISTS

Mini ex: values 10,20,NULL: SUM/COUNT(*)=30/3=10; AVG=SUM/COUNT(col)=30/2=15.
Candidate key search: compute closures for likely determinants; if attr never appears on RHS, it must be in every key; remove attrs to test minimality.
Schema notation: underline PK attrs; write FK(childAttrs → Parent); composite FK must reference a composite candidate/primary key of parent.
NULL in predicates: WHERE x<>5 excludes NULL too, because UNKNOWN is not TRUE. Use x IS NULL or explicit OR when NULL should be included.
Mini ex: FK (deptId) referencing Dept(deptId): many employees may share same deptId; uniqueness would incorrectly force 1:1.
PK/FK DDL: declare parent first or defer FK; composite FK syntax FOREIGN KEY(a,b) REFERENCES Parent(a,b).
Bag semantics: SQL SELECT may return duplicates; relational algebra projection is set projection unless bag RA specified.

W4 RA AND SQL

RA ops: selection σ, projection π, rename ρ, union/intersection/difference, Cartesian product, join, division.
RA query recipe: identify target attrs → build joins by FK path → apply selections early → project last; for "never/no" use set difference or NOT EXISTS.
σ is WHERE; π is SELECT list; join condition goes in ON/WHERE; projection may remove attrs needed for later joins.
Joins: theta/equi/natural; outer joins preserve unmatched rows with NULLs; natural joins all same-named attrs, dangerous after schema drift.
SQL order: FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY.
Equivalence trap: $\pi_{A,C}(\sigma_{D=2}(R \bowtie_{A=E} S))$ not equiv to $\pi_{A,C}(R) \bowtie_{A=E} \sigma_{D=2}(S)$ because E absent after projection unless kept.

English	RA/SQL pattern	Pitfall
both	intersect/self-join	duplicates
never	all - matching	NULLs
highest	>= ALL or max subq	ties
for all	division/NOT EXISTS	double negation

Mini SQL: projects with max staff: GROUP BY project HAVING COUNT(*) >= ALL(SELECT COUNT(*) FROM A GROUP BY project).
Division via NOT EXISTS: return x such that no required y lacks a matching (x,y). Template: NOT EXISTS(R y WHERE NOT EXISTS(S xy)).
RA equivalences: selections commute; projections cascade; join commutative/associative; push σ below join if predicate local; keep join attrs under π.
Outer join warning: applying WHERE predicate to nullable outer side after LEFT JOIN can turn it into inner-join behavior; put predicate in ON if preserving unmatched rows.
Mini ex: cities affected before 1995 and after 2020 = idsBefore ∩ idsAfter; "never affected" = allCityIds - affectedCityIds.
Rename ρ: needed for self-joins or when attrs collide; SQL uses aliases, e.g. Employee e JOIN Employee m.
Natural join danger: joins every same-name attr; if unrelated names match, answer silently changes. Prefer explicit JOIN ON in SQL.

W5 ADVANCED SQL

Aggregation: GROUP BY forms groups; HAVING filters groups; non-aggregated SELECT attrs must be grouped; COUNT(DISTINCT x) for uniqueness.
Nested query types: scalar subquery one value; IN set membership; EXISTS correlated existence; NOT EXISTS robust for anti-join/universal queries.
Universal: students taking all required units = NOT EXISTS(required unit r WHERE NOT EXISTS(enrolment e matching student,r)).
Set ops: UNION removes duplicates, UNION ALL keeps; INTERSECT/EXCEPT are set-level; SQL bags by default.
Views: logical abstraction/security; updatable only for simple SPJ over one base table w/o aggregates/distinct/group.
Trigger vs assertion: assertion checks global condition declaratively; trigger reacts to event/time/level and can maintain derived data or enforce cross-table rules.

Need	Use	Reason
tie max	>=ALL	keeps ties
absence	NOT EXISTS	NULL safe
per group max	HAVING	after grouping
recursive?	CTE	if supported

Mini ex: DELETE parent with child rows: reject/cascade/set null/default; implement by FK action or AFTER DELETE trigger.
Top-k/ties: ORDER BY ... LIMIT may drop ties; max subquery or RANK/DENSE_RANK keeps ties when exam asks "highest".
Window fn: computes over partitions without collapsing rows; e.g. COUNT(*) OVER(PARTITION BY projectId). Use only if allowed/covered.
GROUP BY trap: SELECT projectId,title,COUNT(*) GROUP BY projectId is invalid unless title functionally determined or grouped.
Mini ex: at least two units passed: GROUP BY sid HAVING COUNT(DISTINCT unitId)>=2 after WHERE grade>=50.
Correlated max: WHERE salary >= ALL(SELECT salary FROM Emp e2 WHERE e2.dept=e.dept) returns dept-wise maxima incl ties.
COUNT subtlety: COUNT(col) ignores NULL; COUNT(DISTINCT col) ignores NULL and duplicates; COUNT(*) ignores neither row nor NULL cols.

W6 INTEGRITY

Functional integrity: keys, UNIQUE, CHECK, NOT NULL, FK, assertions, triggers; choose simplest declarative constraint first.
When check RI: child insert/update of FK; parent delete/update of referenced key. Parent insert usually cannot violate existing child FK.
FK references candidate key; FK columns can be non-unique and nullable unless constrained. Entity integrity only applies to primary key attrs.
Actions: RESTRICT/NO ACTION reject; CASCADE propagate; SET NULL/DEFAULT replace; business rule decides.
Trigger anatomy: timing BEFORE/AFTER/INSTEAD OF; event INSERT/UPDATE/DELETE; granularity ROW/STATEMENT; transition vars OLD/NEW.
Row vs statement: row fires per affected row; statement fires once for statement, useful when many child rows may be affected. BEFORE validates/modifies NEW; AFTER maintains dependent effects.

Event	Must check	Why
insert child	FK exists/null	no orphan
delete parent	child refs	orphan risk
insert parent	usually no	adds target
update PK/FK	both sides	broken refs

Assertion English pattern: NOT EXISTS bad rows means "every relevant row satisfies condition"; translate the negated violation, not SQL syntax.
Constraint locality: CHECK sees one row; FK sees parent/child rows; ASSERTION sees whole DB; TRIGGER can query/update multiple tables.
Trigger choice: BEFORE ROW validates/canonicalises each NEW tuple; AFTER ROW audits each change; AFTER STATEMENT enforces set-level maintenance after all rows affected.
Deferred constraints: some DBMSs allow checking at commit, useful for cyclic FKs or batch updates; immediate checking validates each statement.
Mini ex: "Every flight has a booked seat" cannot be simple CHECK on Flight; needs assertion/trigger/query over Reservation.
Candidate vs primary: FK may reference any candidate key declared UNIQUE/PK, not just the primary key in theory/SQL.
Cascades risk: cascading delete can remove many rows; in answers mention business semantics and transaction atomicity.

W7 APP DEVELOPMENT

DB app: program issues SQL via driver/API, binds parameters, maps result sets, commits/rollbacks tx, handles errors and security.
Prepared statements: parse once, bind values, avoid injection, improve plan reuse. Never concatenate user input into SQL.
SQL injection defence: placeholders + least privilege + validation; escaping alone is fragile.
Architecture: presentation/service/DAO; connection pooling; transaction boundary at business operation; ACID error path must rollback.
ORM tradeoff: productivity and object mapping vs hidden N+1 queries, poor joins, leaky transactions; inspect generated SQL.
Concurrency in apps: keep tx short; consistent isolation; retry deadlocks; idempotent writes for retry safety.

Problem	Symptom	Fix
N+1 queries	many selects	eager join/batch
lost update	overwrite	locks/version
dirty read	uncommitted	isolation
injection	tautology input	bind params

Mini ex: transfer money: BEGIN; debit; credit; if both ok COMMIT else ROLLBACK; never commit between debit and credit.
Isolation in apps: READ COMMITTED avoids dirty reads; SERIALIZABLE strongest but may abort/retry; choose based on business invariant risk.
Error path: catch DB exception, rollback, release/return connection, surface safe error; never keep failed tx open in pooled connection.
Parameter binding: values bind, identifiers usually cannot; whitelist table/column names if dynamic SQL must choose identifiers.
Mini ex: capacity check needs lock or serializable tx; otherwise two concurrent inserts can both see one remaining seat.
Retry loop: for serialization failure/deadlock: rollback, wait/backoff, reread state, retry whole business tx.
Audit trigger: store OLD/NEW values, user, timestamp; use AFTER ROW so the base modification has succeeded.

W8 NORMALISATION

FD: $X \to Y$ means same X values imply same Y values; superkey determines all attrs; candidate key=minimal superkey.
Closure: start $X^+ = X$; repeatedly add RHS when LHS subset of closure; if closure all attrs then X is superkey.
BCNF: for every nontrivial FD $X \to Y$, X is superkey. 3NF allows RHS prime attrs. Every 2-attr relation is BCNF.
Anomalies: update inconsistency, insertion impossible w/o unrelated fact, deletion loses fact; decomposition separates facts.
BCNF decomp: find violation $X \to Y$; replace R by $R_1 = X \cup Y$, $R_2 = R - (Y - X)$; lossless because intersection X determines R_1.
Minimal cover: split RHS, remove extraneous attrs, remove redundant FDs; used for 3NF synthesis.

NF	Rule	Allows
1NF	atomic attrs	no repeating group
2NF	no partial dep	for composite keys
3NF	X key or A prime	dep preserving
BCNF	X superkey	stronger

Mini ex: R(A,B,C,D), F=A→B, B→C. A+=A,B,C; not key unless D included; AD is key; B→C violates BCNF.
Armstrong axioms: reflexivity, augmentation, transitivity; derived rules union/decomposition/pseudotransitivity speed FD reasoning.
Minimal cover: split RHS; remove extraneous LHS attrs by closure test; remove redundant FDs; then synthesize 3NF relations from LHS→RHS groups.
Dependency preservation: all original FDs enforceable without joining decomposed tables; BCNF may lose it, 3NF synthesis usually preserves.
Mini ex: R(A,B,C), F=A→B,B→A,B→C; A and B are keys, so BCNF holds for all listed nontrivial FDs.
Prime attrs: attr appearing in at least one candidate key; 3NF exception uses prime RHS, not arbitrary RHS.
Partial dependency: non-prime attr depends on proper subset of composite key; violates 2NF and often signals table split.

W9 TRANSACTIONS

ACID: atomicity all-or-nothing; consistency preserves constraints; isolation appears serial; durability survives crash.
Anomalies: dirty read, non-repeatable read, phantom, lost update; isolation levels trade correctness/performance.
Conflict serializability: nodes tx; edge $T_i \to T_j$ if conflicting ops same item, different tx, at least one write, i before j; acyclic iff conflict serializable.
Equivalent serial schedules are topological orders of precedence graph. Cycle means not conflict serializable.
Locks: S for read, X for write; compatibility S/S only; 2PL growing then shrinking; strict 2PL holds X to commit for recoverability.
Deadlock: wait-for graph cycle; handle timeout/detection+abort; starvation avoided by fair scheduling.

Pair	Conflict?	Edge
$R_i(X), R_j(X)$	no	none
$R_i(X), W_j(X)$	yes	$i \to j$
$W_i(X), R_j(X)$	yes	$i \to j$
$W_i(X), W_j(X)$	yes	$i \to j$

Mini ex: R1(X),W2(X),W1(Y),R2(Y) gives edges T1→T2 and T2→T1, so cycle.
Recoverability: if T_j reads from T_i, then T_i should commit before T_j commits; strict schedules avoid cascading aborts.
Isolation levels: read uncommitted (dirty possible), read committed, repeatable read, serializable; SQL implementations vary.
2PL test: once a tx releases any lock, it may not acquire new locks. Strict 2PL also keeps X locks to commit/abort.
Mini ex: lost update: T1 reads x, T2 reads x, both write x based on stale value; fix X locks or optimistic version check.
Serial vs serializable: serial has no interleaving; serializable may interleave but equivalent to some serial order.
View serializability: broader than conflict serializability; course exams usually use conflict graph unless stated otherwise.

W10 INTEGRATION

Mid-sem integration: W1–W9 questions often mix ER/RM/SQL/FD/tx; always state assumptions and constraints explicitly.
Answering pattern: identify schema objects; mark PK/FK/FD; choose operator/constraint; write concise justification; test edge case (NULL, duplicate, cascade, cycle).
For "which statements true", check exact quantifiers: every/some/at most/exactly; most wrong choices fail on min-cardinality or uniqueness.
Common final shapes: MCQ definitions, ER-to-RM, RA/SQL translation, assertion English, FK action, closure/BCNF proof, serializability graph, storage/join cost.
Proof style: for 2-attr BCNF, any nontrivial FD is A→B or B→A; determinant determines all attrs hence is key; otherwise no nontrivial FD.
Plain English assertions: convert NOT EXISTS violation into universal sentence: "Every flight has at least one booked reservation."

Cue	Likely topic	Check
max/min	SQL agg	ties
booked none	assertion	double neg
April+age	RA/SQL	date overlap
buffer pages	sort/join cost	ceilings

Mini checklist before final answer: PKs underlined? FK arrows? RA attrs retained? SQL GROUP valid? graph edges all conflicts? cost units pages vs tuples?
Mark allocation: 2 marks usually definition/choice+reason; 4–6 marks need method+working; 8+ marks need schema/query plus constraints and assumptions.
Assumption style: if diagram/text ambiguous, state minimal assumption before answer; do not invent extra entities/attrs not in UoD.
High-frequency COMP9120 final: ER/EER mapping, RA/SQL, FK/trigger/assertion, FD+BCNF, conflict graph, external sort/storage/join cost.
Mini ex: "plain English" answer should be one business sentence, not a line-by-line SQL paraphrase.
MCQ elimination: test counterexamples with NULL, duplicate key, optional participation, and tie maxima.
Show working: for cost/closure/graph, final answer alone often loses marks; include runs, closure steps, or edges.

USYD各科往届题库

W11 STORAGE INDEXING

Storage: page/block is I/O unit; record size and page usable bytes determine tuples/page; file pages = $\lceil N/\lfloor usable/record \rfloor \rceil$ adjusted for fill factor.
Usable page bytes = page size - header. With occupancy f, effective records/page = $\lfloor f \cdot usable/recordSize \rfloor$.
Files: heap fast insert, scan for search; sorted supports range/order but costly insert; hashed supports equality, poor range; clustered stores data near key order.
Indexes: primary/secondary, clustered/unclustered, dense/sparse; B+ tree supports equality+range, hash equality only.
B+ tree lookup: root-to-leaf I/Os + data page I/Os; range scan leaf chain + data pages; clustering dramatically reduces range data I/O.
Index choice: equality predicate high selectivity -> B+ or hash; range/order -> B+; low selectivity -> scan may win; composite index prefix matters.

Index	Best	Weakness
B+ tree	range/order	eash update split
Hash	equality	no range
Clustered	range data	one per table
Unclustered	point lookup	many random I/O

Mini ex: 400k rows, 32B each, page 4096B, header 196B, fill .9: floor(.9*(3900)/32)=109 rows/page; pages=ceil(400000/109)=3670.
B+ tree height: fanout $F \approx \lfloor pageSize/entrySize \rfloor$; leaves store search key+RID or data ptr; height grows logarithmically.
Composite index: (A,B,C) helps predicates on A, A+B, A+B+C; B-only predicate usually cannot use prefix efficiently.
Clustered range: qualifying records near each other, so data page cost close to range pages; unclustered range may fetch one page per tuple.
Mini ex: selectivity 40% on unclustered index likely scan; selectivity 0.1% with equality likely index.
Index maintenance: every insert/delete/update of indexed attr updates index; indexes speed reads but slow writes and consume pages.
Sparse index precondition: data file sorted/clustered on search key; one entry per page/range, not suitable for arbitrary heap lookup.
RID cost: index leaf points to record id/page; clustered range has sequential-ish RID locality, unclustered random RIDs.
Mini ex: 3-level B+ tree equality lookup often costs root+internal+leaf plus data page; cache root can reduce real I/O.

W12 QUERY OPT

Query evaluation: parse/rewrite/optimise/execute; algebra equivalences push selections/projections, reorder joins, choose access paths and algorithms.
External sort: with B buffers, pass0 creates $\lceil N/B \rceil$ runs; merge fan-in B-1; total passes $1 + \lceil \log_{B-1} \lceil N/B \rceil \rceil$; I/O about $2N$ per pass.
Two-pass sort condition: $\lceil N/B \rceil \le B-1$ roughly $N \le B(B-1)$; solve for min B by trial.
Join algorithms: tuple nested loop terrible; page nested loop; block nested loop $B(R) + \lceil B(R)/(M-2) \rceil B(S)$; index nested loop good if inner indexed; sort-merge/hybrid hash for large equi-join.
Cost model: pages dominate; cardinality/selectivity estimate output; choose smaller outer for nested loops, but clustering/index can override.
Projection pushdown warning: do not drop join attrs before join; selection pushdown safe if predicate references one input.

Alg	Cost sketch	Use
page NLJ	$B_R + B_R B_S$	small outer
block NLJ	$B_R + \lceil B_R/(M-2) \rceil B_S$	buffers help
sort-merge	sort R+sort S+$B_R + B_S$	ordered/equi
hash join	approx $3(B_R + B_S)$	equi, enough mem

Mini ex: N=1600 pages, B=4: pass0 400 runs, merge fan-in3, merge passes $\lceil \log_3 400 \rceil = 6$, total passes 7, I/O approx $2N \cdot 7$.
Plan comparison: estimate cardinalities after filters, choose access paths, choose join order/algorithm, account for sort/order/group requirements.
Selectivity heuristics: equality on key $\approx 1/N$ tuple; equality on attr with V distinct $\approx 1/V$; range depends on histogram/assumption.
Join order: push most selective filters early; join smaller/intermediate results first, unless an index nested-loop on larger outer is cheaper.
Mini ex: block NLJ with R=2000 pages, S=500, M=52: cost R outer = 2000+ceil(2000/50)*500=22000.
Projection pushdown: keep output attrs plus join/filter attrs; dropping required attrs changes query or prevents later operators.
Sort reuse: ORDER BY/GROUP BY/merge join may reuse an existing sorted order or clustered B+ tree order, reducing explicit sort cost.

USYD Past Papers · vx: courses1 · Buyer: USYD Past Papers